



Introducción a Java

Sintaxis

La sintaxis de un lenguaje de programación es el conjunto de reglas y parámetros que deben seguirse al redactar el código de los programas para que pasen a ser considerados como correctos para ese lenguaje.

La sintaxis dicta cómo se definen las combinaciones de símbolos que se consideran declaraciones o expresiones correctamente estructuradas, es decir cómo se utilizan las palabras clave y los operadores, y donde deben aparecer los símbolos de puntuación.

```
/**
 * This example is much more readable than Compact.java.
 */

public class Readable
{
    public static void main(String[] args)
    {
        int shares = 220;
        double averagePrice = 14.67;

        System.out.println("There were " + shares +
                           " shares sold at $" +
                           averagePrice + " per share.");
    }
}
```

Veamos un programa en Java de ejemplo e identifiquemos una instancia de cada uno de estos elementos.

Ejemplo 1:

```
public class Nomina {
    public static void main(String[] args)
    {
        int horas = 40;
        double sueldoBruto, salario = 25.0;
        sueldoBruto = horas * salario;
        System.out.println("Tu sueldo bruto es $" + sueldoBruto);
    }
}
```



Se clasifican en:

Palabras

Este nivel léxico determina como los caracteres forman tokens. Son las unidades mínimas con significado que se obtienen a partir de una secuencia de caracteres del código fuente.

Frases:

Este nivel gramatical determina como los tokens se combinan para formar frases validas dentro del lenguaje.

Contexto:

Este nivel contextual determina las convenciones de nomenclatura, como a que variables u objetos se refieren los identificadores, así como la validez y la compatibilidad de los tipos.

Comentarios

Java permite incluir un **comentarios** en el código fuente para explicar o documentar el funcionamiento del programa.

Los comentarios son ignorados por el compilador y solo sirven para quienes leen el código.

Java soporta tres estilos de comentarios. El que se muestra en la parte superior del programa se denomina **comentario de varias líneas**

```
/* Este es un programa simple de Java  
Nombra este archivo Clase.java  
*/
```

Este tipo de comentario debe comenzar con **/*** y **terminar con */** y se usa para explicaciones largas.

La siguiente linea es un ejemplo de un **comentario de una sola linea**:



```
// Un programa en Java inicia con una llamada al método main()
```

Comienza con // y se usa para descripciones breves.

La siguiente línea es un ejemplo de un **comentario al final de una instrucción**

```
int miVar1; // declaración de una variable
```

Se puede poner un comentario de una sola línea después de una instrucción de código.

Palabras clave (palabras reservadas)

Las palabras clave son términos reservados por el lenguaje y no se pueden usar para nada más que para el propósito específico para el cual fueron definidas.

El programador puede crear nombres propios, pero no puede usar palabras claves como identificadores (variables, clases o métodos).

A continuación, se muestra la tabla de las palabras reservadas.

abstract	const	final	int	public	throw
assert	continue	finally	interface	return	throws
boolean	default	float	long	short	transient
break	do	for	native	static	true
byte	double	goto	new	strictfp	try
case	else	if	null	super	void
catch	enum	implements	package	switch	volatile
char	extends	import	private	synchronized	while
class	false	instanceof	protected	this	

Tabla 1: Palabras reservadas de Java

Líneas y sentencias

Línea: Es una línea del programa. Puede contener código o estar en blanco para mejorar legibilidad.

Sentencia: Es una instrucción completa que indica a la computadora realizar una acción. Puede ocupar una o varias líneas y suelen terminar con ;



```
System.out.println("Tu sueldo bruto es $" + sueldoBruto);
```

La sentencia anterior hace que la computadora muestre un mensaje en la pantalla. Las sentencias pueden ser una combinación de palabras clave, operadores y nombres definidos por el programador.

Variables

Una **variable** es un espacio de memoria con un nombre donde se almacenan datos.

Los datos de una variable se guardan en la RAM.

Las variables representan ubicaciones de memoria mediante nombres simbólicos creados por el programador.

Declaración de una variable

En Java, todas las variables deben declararse antes de usarse. La declaración indica el tipo de dato y el nombre de la variable.

Forma general:

Tipo_dato nombre_variable;

aquí, **tipo_dato** especifica el tipo de dato de la variable que se declara y **nombre_variable** o identificador es el nombre de la variable. Observe que las declaraciones de variable terminan con punto y coma.

Identificadores

Un identificador es un nombre creado por el programador para representar variables, clases o métodos. No pueden ser palabras reservadas. Deben ser claros y descriptivos.

Reglas básicas:

- Inician con una letra, _ o \$.



- Pueden contener letras, números, _ o \$.
- No admiten espacios,
- Distinguen mayúsculas y minúsculas.
- Se recomienda iniciar variables con minúsculas y usar camelCase.
- Evitar el uso del carácter.

Algunos ejemplos de identificadores válidos son:

TempProm	contador	a4	\$test	este_esta_bien
----------	----------	----	--------	----------------

Nombres de identificadores inválidos:

2contador	temp-alta	mal/ok
-----------	-----------	--------

Literales

Un literal es un valor escrito directamente en el código de programa. Representa datos como números, caracteres o cadenas y pueden usarse donde se permita un valor de ese tipo.

Ejemplos:

100 (entero), 98.6 (flotante), 'X' (carácter), "Esto es una prueba" (cadena).

Método main()

El método **main()** es el punto de inicio de una aplicación Java.

Se ejecuta automáticamente al iniciar el programa. Recibe un parámetro: `String[] args`, un arreglo de cadenas que almacena los argumentos de la línea de comandos.

```
public static void main(String[] args)
{
}
}
```



Tipos de datos

Los tipos de datos definen el tipo de información que puede almacenar una variable. Java es un lenguaje fuertemente tipado, lo que significa que el compilador verifica la compatibilidad entre tipos y no permite operaciones inválidas.

Tipos primitivos de Java

Java contiene dos categorías generales de tipos de datos integrados: orientados a objetos y no orientados a objetos.

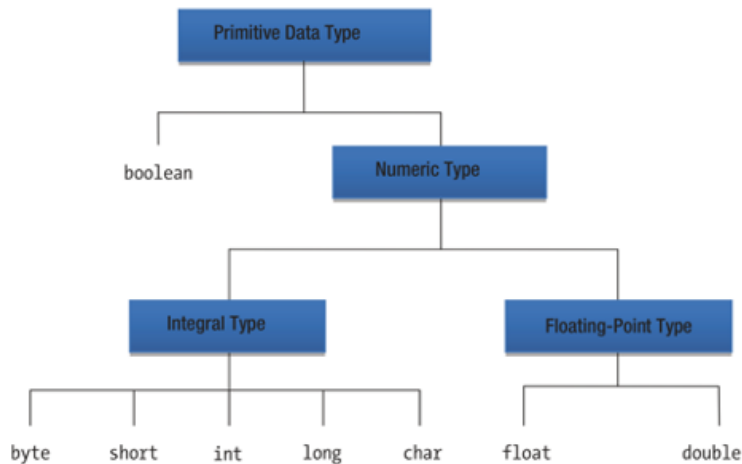


Figura 1: Lista de datos primitivos de Java divididos por categorías

En Java hay ocho tipos de datos primitivos (también llamados elementales o simples), que se muestran en la tabla.

Tipo	Significado
boolean	Representa valores: verdadero/falso
byte	Enteros de 8 bits
char	Carácter
double	Punto flotante de doble precisión
float	Punto flotante de una precisión
int	Enteros
long	Entero long
short	Entero short

Tabla 2: Datos primitivos de Java Build-in



El termino primitivo se usa aquí para indicar que estos tipos no son objetos en un sentido orientado a objetos, sino valores binarios normales. Todos los demás tipos de datos en Java se construyen a partir de estos tipos de datos primitivos.

Java especifica estrictamente un rango y un comportamiento para cada tipo primitivo, que todas las implementaciones de Java Virtual Machine debe admitir.

Enteros

Java define cuatro tipos de enteros: **byte**, **short**, **int** y **long**

Tipo	Ancho en bits	Significado
byte	8	-128 a 127
short	26	-32,768 a 32,767
int	32	-2,147,483,648 a 2,147,483,647
long	64	-9,223,372,036,854,775,808 9,223,372,036,854,775,807

Tabla 2: Tipos de datos

Todos los tipos **enteros** son valores positivos y negativos con signo. Java no admite enteros sin signo (solo positivos).

Tipos de punto flotante

Los tipos de punto flotante pueden representar números que tienen componentes fraccionarios. Hay dos tipos de coma flotante, **float** y **double**, que representan números de precisión simple y doble respectivamente. El tipo **float** tiene 32 bits de ancho y el tipo **double** 64 bits de ancho. De los dos el double es el más utilizado y muchas de las funciones matemáticas de la biblioteca de clases de Java utilizan estos valores dobles.

Caracteres

En Java, los caracteres no son cantidades de 8 bits como los son en otros lenguajes. En cambio, Java usa Unicode. Unicode define un conjunto de caracteres que puede representar todos los caracteres que puede representar todos los caracteres que se encuentran en todos los idiomas humanos. En Java, char es un tipo de 16 bits sin firmar que tiene un rango de 0 a 65 535. El conjunto de caracteres ASCII estándar de 8 bits es un subconjunto de Unicode y varía de 0 a 127. Por lo tanto, los caracteres ASCII siguen siendo caracteres válidos de Java.



Ejemplo:

```
char a = 'A';  
//error  
  
char b = "B"; //<-esto no es un char, es un String
```

Boolean

Este tipo de dato representa solo dos valores true/false se utilizan para evaluar condiciones lógicas.

Ejemplo:

```
boolean x = false;  
//no quiere decir falso, quiere decir que x es igual a cero y no compila  
/*if (x == 0)  
{  
    ...  
}*/  
//deberia ser  
if(x == false)  
{  
    ...  
}
```

Inicializar una variable

En general, debe dar un valor a una variable antes de usarla. Esto se denomina sentencia de asignación. El signo igual es un operador que almacena el valor que se encuentra a su derecha en la variable nombrada a la izquierda. Otra forma es dándole un valor inicial cuando se declara.

La forma general de inicialización se muestra aquí:

tipo var = valor;

Aquí el valor es el valor que se le da a **var** cuando se crea la variable. El valor debe ser compatible con el tipo especificado. Aquí hay unos ejemplos:

```
int cuenta = 10; // se le da un valor inicial de 10 a cuenta  
char ch = 'X'; // se inicializa a ch con una letra X  
float f = 1.2F; // f es inicializada con 1.2  
boolean fin = true;  
//constantes en java se usa fina
```




Al declarar dos o más variables del mismo tipo usando una lista separada por comas, puede dar un valor inicial a una o más de esas variables. Por ejemplo:

```
int a, b = 8, c = 19, d; // b y c tienen inicializaciones
```

Ejemplo 2: Tipos de datos primitivos

```
// Desplegar los diferentes tipos de datos.
public class TiposDatosPrueba {
    public static void main(String[] args) {
        boolean b = false;
        char c = 'R';
        byte j = 127;
        short k = 32767;
        int m = 2147483647;
        long n = 9223372036854775807L;
        float x = 3.14159265F;
        double y = 3.141592653589793238;
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("j = " + j);
        System.out.println("k = " + k);
        System.out.println("m = " + m);
        System.out.println("n = " + n);
        System.out.println("x = " + x);
        System.out.println("y = " + y);
    }
}
```

Operadores

Un operador es un símbolo que le dice al compilador que realice una manipulación matemática o lógica específica. Es decir, realizan operaciones sobre los elementos de datos, conocidos como operandos. Java tiene cuatro clases generales de operadores: aritméticos, bit a bit, relacional y lógico. Java también define algunos operadores adicionales que manejan ciertas situaciones especiales.

Operadores aritméticos

Java define los siguientes operadores aritméticos.

Operador	Significado
+	Suma (solo sumas unarias)
-	Resta (solo restas unarias)
*	Multiplicación



/	División
%	Modulo
++	Incremento
--	Decremento

Tabla 3: Operadores aritméticos

Los operadores +, -, * y / funcionan de la misma manera en Java que en cualquier otro lenguaje informático (o algebra, para el caso). Estos se pueden aplicar a cualquier tipo de datos numéricos integrados. También se pueden usar en objetos de tipo char.

Primero, recuerde que cuando / se aplica a un número entero, cualquier resto se truncará; por ejemplo, 10/3 será igual a 3 en la división de enteros. Puede obtener el resto de esta división utilizando el operador de módulo %. Da el resto de una división entera. Por ejemplo, 10 % 3 es 1. En Java, el % se puede aplicar tanto a tipos enteros como de coma flotante. Por lo tanto, 10,0 % 3,0 también es 1. El siguiente programa demuestra el operador de módulo.

Ejemplo 3: Prueba de operadores aritméticos

```
// Demostración del operador modulo %.  
public class ModuloDemo {  
    public static void main(String[] args) {  
        int iresult, iresiduo;  
        double dresult, dresiduo;  
        iresult = 10 / 3;  
        iresiduo = 10 % 3;  
        dresult = 10.0 / 3.0;  
        dresiduo = 10.0 % 3.0;  
        System.out.println("Resultado y residuo de 10 / 3: " + iresult + " " +  
iresiduo);  
        System.out.println("Resultado y residuo de 10.0 / 3.0: " + dresult + " " +  
dresiduo);  
    }  
}
```

Operadores relacionales y lógicos

Un **operador relacional** y **operador lógico**, relacional se refiere a las relaciones que los valores pueden tener entre sí, y lógico se refiere a las formas en que los valores verdaderos y falsos se pueden conectar entre sí. Dado que los operadores relacionales producen resultados verdaderos o falsos, a menudo funcionan con los operadores lógicos.

Operadores relacionales



Operador	Significado
==	Igual que
!=	Diferente de
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que

Tabla 4: Operadores relacionales

Operadores lógicos

Operador	Significado
&	AND
	OR
	XOR (OR exclusiva)
	Corto circuito OR
&&	Corto circuito AND
!	NOT

Tabla 5: Datos primitivos de Java Build-in

¡En Java, todos los objetos se pueden comparar en cuanto a igualdad o desigualdad usando == y !=. Sin embargo, los operadores de comparación <, >, <= o >=, solo se pueden aplicar a aquellos tipos que admitan una relación de orden. Por lo tanto, todos los operadores relacionales se pueden aplicar a todos los tipos numéricos y al tipo char. Sin embargo, los valores de tipo booleano solo se pueden comparar por igualdad o desigualdad, ya que los valores verdadero y falso no están ordenados.

Para los operadores lógicos, los operandos deben ser de tipo booleano, y el resultado de una operación lógica es de tipo booleano. Los operadores lógicos &, |, ^, y ! admiten las operaciones lógicas AND, OR, XOR y NOT, de acuerdo con la siguiente tabla.

p	q	p & q	p q	p ^ q	!p
False	False	False	False	False	True
True	False	False	True	True	False
False	True	False	True	True	True
True	True	True	True	False	False



Tabla 6: Tablas de verdad

El siguiente ejemplo muestra varios de los operadores relacionales y lógicos.

Ejemplo 4. Prueba de operadores relacionales y lógicos

```
/// Demostración de los operadores relacionales y logicos
public class PruebaOperadoresRelacionales {
    public static void main(String[] args) {
        int i, j;
        boolean b1, b2;
        i = 10;
        j = 11;
        if (i < j)
            System.out.println("i < j");
        if (i <= j)
            System.out.println("i <= j");
        if (i != j)
            System.out.println("i != j");
        if (i == j)
            System.out.println("esto no se puede ejecutar");
        if (i >= j)
            System.out.println("esto no se puede ejecutar");
        if (i > j)
            System.out.println("esto no se puede ejecutar");
        b1 = true;
        b2 = false;
        if (b1 & b2)
            System.out.println("esto no se puede ejecutar");
        if (!(b1 & b2))
            System.out.println("!(b1 & b2) es true");
        if (b1 | b2)
            System.out.println("b1 | b2 es true");
        if (b1 ^ b2)
            System.out.println("b1 ^ b2 es true");
    }
}
```

Operadores de asignación

El operador de asignación es el único signo igual, =. Este operador funciona en Java como lo hace cualquier otro lenguaje. Tiene esta forma general.

var = expresión;

Aquí, el tipo *var* debe ser compatible con el tipo de *expresión*.

El operador de asignación tiene un atributo interesante con el que quizás no esté familiarizado; permite crear una cadena de asignaciones. Por ejemplo



```
int x, y, z;  
x = y = z = 100; // inicializar x, y, y z a 100
```

Este código establece las variables x, y y z en 100 usando una sola instrucción. Esto funciona porque = es un operador que produce el valor de la expresión de la derecha. Así, el valor de z=100 es 100, que luego se asigna a y, que a su vez se asigna a x.

Los operadores de asignación combinados combinan el operador de asignación con los operadores aritméticos. Muy a menudo los programas tienen instrucciones de la siguiente forma:

```
x = x + 1;
```

En el lado derecho del operador de asignación, se suma 1 a x. El resultado se asigna a x, reemplazando el valor que estaba allí anteriormente

```
x += 1;
```

El par de operadores += le dice al compilador que asigne a x el valor de x mas 1. Aquí hay otro ejemplo.

```
x = x - 1;
```

se puede escribir, usando los operadores de asignación de Java, como:

```
x -= 1;
```

El efecto de esta instrucción es que 1 se resta del valor almacenado en la variable x.

Esta abreviatura funcionará para todos los operadores binarios en Java (es decir, aquellos que requieren dos operandos). La forma general es:

Así, los operadores de asignación aritméticos y lógicos abreviados son los siguientes.

Operador	Ejemplo de uso	Equivalente a
+=	x += 5;	x = x + 5;
-=	x -= 2;	x = x - 2;
*=	x *= 10;	x = x * 10;
/=	a /= b;	a = a / b;
%=	c %= 3;	c = c % 3;

Tabla 7: Operadores de asignación

Debido a que estos operadores combinan una operación con una asignación, se los conoce formalmente como operadores de asignación compuestos.

Conversión entre tipos de datos primitivos



Antes de que un valor pueda almacenarse en una variable, el tipo de datos del valor debe ser compatible con el tipo de datos de la variable. Java realiza algunas conversiones entre tipos de datos de forma automática, pero no realiza automáticamente ninguna conversión que pueda provocar la pérdida de datos. Java sigue un conjunto de reglas al evaluar expresiones aritméticas que contienen datos mixtos.

Java es un lenguaje fuertemente tipado. Esto significa que antes de asignar un valor a una variable, Java comprueba los tipos de datos de la variable y del valor que se le asigna para determinar si son compatibles. Por ejemplo, las siguientes afirmaciones.

```
int x;  
double y;  
x = 10;  
y = x; // asigna a un entero hacia un double
```

Esta sentencia de asignación, que almacena un entero en un double, funcionara sin problemas. Sin embargo, debido a la estricta verificación de tipos de Java, no todos los tipos son compatibles y, por lo tanto, no todas las conversiones de tipos están implícitamente permitidas. Por ejemplo, booleano e int no son compatibles.

Cuando se asigna un tipo de datos a otro tipo de variable, se realizará una conversión de tipo automática si

- Los dos tipos son compatibles.
- El tipo de destino es más grande que el tipo de origen.

Cuando se cumplen estas dos condiciones, tiene lugar una conversión ampliada.

Operador de conversión (cast)

El **operador de conversión (cast)** permite convertir manualmente el tipo de un valor, incluso cuando la conversión no es automática. Es un operador unario que se escribe como el tipo de dato entre paréntesis y se colocan antes de la expresión a convertir.

variable = (tipo_de_dato) expresión

Aquí, *tipo_de_dato* especifica el tipo deseado para convertir la expresión especificada.



Por ejemplo, al convertir el resultado de x/y de double a int, se usa

```
double x, y;  
// ...  
(int) (x / y)
```

Los paréntesis son importantes para que la conversión se aplique al resultado completo y no solo a una variable. Este cast es necesario porque no existe conversión automática de double a int.

Sentencia	Descripción
<code>x = (long) 3.7;</code>	El operador de conversión se aplica a la expresión 3.7. El operador devuelve el valor 3, el cual es asignado a la variable x
<code>numero = (int)72.567;</code>	El operador de conversión se aplica a la expresión 72.567. El operador devuelve 72, que se utiliza para inicializar la variable numero
<code>valor = (float)x;</code>	El operador de conversión regresa el valor en x, convertido a float. El valor convertido se asigna a la variable valor
<code>valor = (byte)numero;</code>	El operador de conversión devuelve el valor en número, convertido a un byte. El valor convertido se asigna a la variable valor

Tabla 8: Ejemplo de uso de operador de conversión

Cuando una conversión implica una conversión de restricción, es posible que se pierda información.

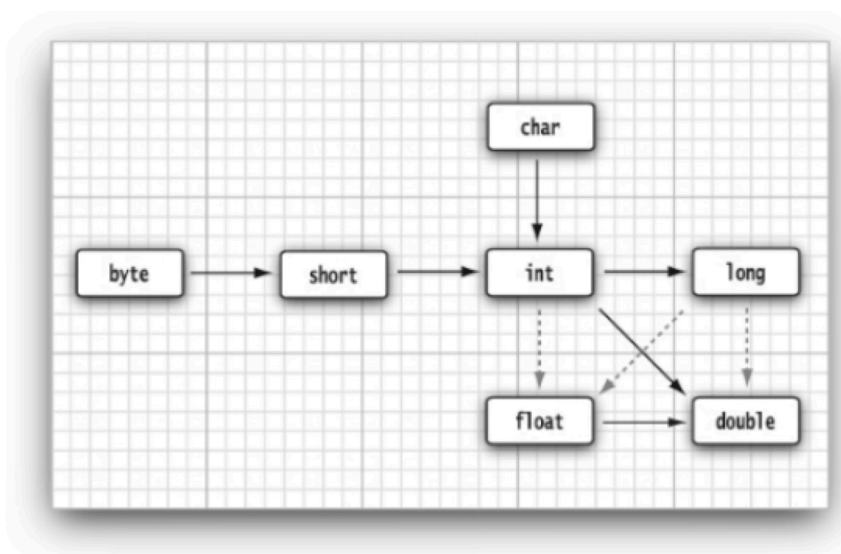


Figure 3.1 Legal conversions between numeric types



Las líneas punteadas es donde hay una pérdida de información.

Ejemplo 5. Demostración de conversión de tipos de datos

```
// Demostración de casting.
public class CastDemo {
    public static void main(String[] args) {
        double x, y;
        byte b;
        int i;
        char ch;
        x = 10.0;
        y = 3.0;
        i = (int) (x / y); // convertir un doble a entero
        System.out.println("La salida de x / y: " + i);
        i = 100;
        b = (byte) i;
        System.out.println("El valor de b: " + b);
        i = 257;
        b = (byte) i;
        b = 88; // Código ASCII para X
        ch = (char) b;
        System.out.println("ch: " + ch);
    }
}
```

Precedencia de operadores

La tabla 2-3 muestra el orden de precedencia de todos los operadores de Java, de mayor a menor. Aunque técnicamente son separadores, [], () y . también pueden actuar como operadores. En esa capacidad, tendrían la más alta precedencia.



Highest						
++ (postfix)	-- (postfix)					
++ (prefix)	-- (prefix)	~	!	+ (unary)	- (unary)	(type-cast)
*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&&						
^						
&&&						
?:						
->						
=	op=					
Lowest						

Tabla 9: Reglas de precedencia de operadores

Secuencia de caracteres escape

En java las constantes de caracteres se escriben entre comillas, pero algunos caracteres especiales (como el retorno de carro o las comillas) no pueden usarse directamente. Para resolver esto, java utiliza secuencias de escape (con barra invertida), que representan estos caracteres especiales y se usan en su lugar, las cuales se muestran en la siguiente Tabla.

Secuencia de escape	Nombre	Descripción
\n	Línea nueva	Avanza el cursor a la siguiente línea para su posterior impresión
\t	Tab horizontal	Hace que el cursor salte a la siguiente tabulación
\b	Retroceso	Hace que el cursor retroceda o se mueva una posición hacia la izquierda
\r	Retorno	Hace que el cursor vaya al principio de la línea actual, no a la siguiente
\\	Barra invertida	Hace que se imprima una barra invertida
\'	Comilla simple	Hace que se imprima una comilla simple
\"	Comilla doble	Hace que se imprima una comilla doble

Tabla 10: Secuencias de escape



Estructuras de Control de Selección

La sentencia de control: if

La **sentencia if** permite crear estructuras de decisión en un programa, haciendo que ciertas instrucciones se ejecuten **solo si una condición booleana es verdadera**. Es la forma más simple de control de flujo y se usa cuando una acción debe realizarse únicamente al cumplirse

La forma completa de la sentencia if simple es:

```
if(condición)
```

```
    sentencias;
```

Donde la **condición** debe ser una expresión booleana. Si la **condición** es **verdadera** se ejecutará la acción (statements o sentencias) de lo contrario se terminará la sentencia **if**. A continuación, se muestra el diagrama de flujo.

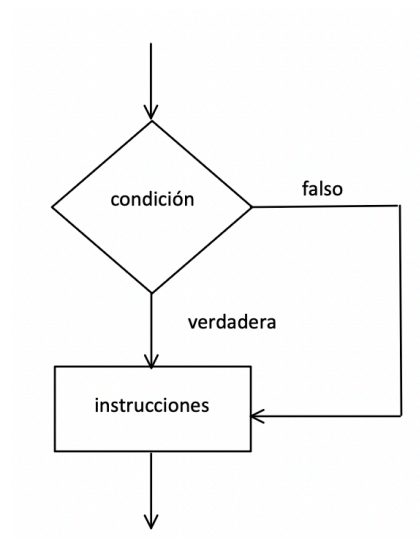


Figura 2: Diagrama de flujo de la sentencia if.

La sentencia if puede ejecutar condicionalmente un grupo de sentencias, siempre que estén encerradas entre llaves.

**Ejemplo 6.** Sentencia if simple

```
/*
    Programa ejemplo sentencia if "Ejemplo3.java".
*/
public class Ejemplo3 {
    public static void main(String[] args) {
        int edad = 20;
        if (num1 > 18) {
            System.out.println("Eres mayor de edad ");
        }
        System.out.println("Terminó la sentencia if ");
    }
}
```

La sentencia de control: if-else

La **sentencia if-else** permite elegir entre dos rutas de ejecución según el resultado de expresión booleana. Si la condición es verdadera, se ejecuta un bloque de sentencias; si es falsa, se ejecuta otro bloque distinto. La forma completa de la sentencia if-else es:

```
if(condición)
    sentencia1 o bloque;
else
    sentencia2 o bloque;
```

Donde la **condición** de **if** y **else** son declaraciones individuales.

La condición siempre debe evaluarse como verdadero/falso, y solo uno de los bloques se ejecuta, nunca ambos. A continuación, se muestra el diagrama de flujo y un ejemplo.

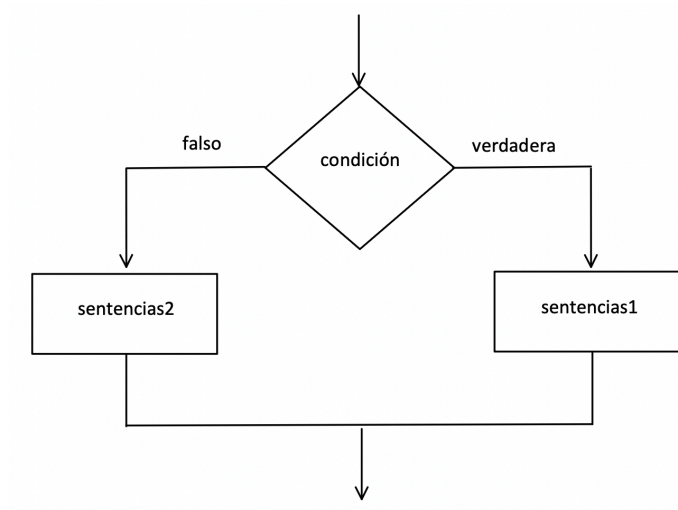


Figura 3: Diagrama de flujo de la sentencia if-else.

Ejemplo 7. Sentencia if -else

```
/*
Programa ejemplo sentencia if "Ejemplo4.java".
*/
public class Ejemplo4 {
    public static void main(String[] args) {
        int num1 = 50;
        int num2 = 5;
        int num3 = 10;
        if (num1 > 50) {
            num2 = num2 + 10;
            System.out.println("El valor de num2 es " + num2);
            num3 = num3 + 10;
            System.out.println("El valor de num2 es " + num3);
        } else {
            num2 = num2 - 10;
            System.out.println("El valor de num2 es " + num2);
            num3 = num3 - 10;
            System.out.println("El valor de num2 es " + num3);
        }
    }
}
```

La sentencia de control: if-else multiple

La **sentencia if-else-if** se utiliza para evaluar múltiples condiciones de forma secuencial y resulta más clara que usar varios if anidados. El programa prueba cada condición booleana desde arriba: cuando una se evalúa como verdadera, se ejecuta su bloque de sentencias y se ignoran las



demás. Si ninguna condición es verdadera, se ejecuta la cláusula else final, que actúa como caso por defecto. La forma completa de la sentencia if-else multiple es:

```
if(condicion1)

    sentencia1;

else if(condicion2)

    sentencia2;

else

    sentenciaN;
```

Ejemplo 8. Sentencia if -else-multiple

```
/*
Programa ejemplo sentencia if "Ejemplo5.java".*/
public class Ejemplo5 {
    public static void main(String[] args) {
        int edad = 10;
        if (edad > 50) {
            System.out.println("Niño ");
        }
        else if (num1 < 18) {
            System.out.println("Adolescente");
        }
        else {
            System.out.println("Adulto ");
        }
    }
}
```

La sentencia de control: if-anidado

A veces, una declaración if debe estar anidada dentro de otra declaración if. Para probar más de una condición, una declaración if puede estar anidada dentro de otra declaración if.

```
if(condicion1) {
    if(condicion2)
        sentencia1;
    else statement2; {
        sentnecia3;
```



```
        else; {  
            sentencia3;  
        }  
    }
```

Las **condiciones** deben ser expresiones booleanas que se evalúan como verdadero o falso. Si la condición1 es verdadera, se entra al if y se evalúa el if interno para decidir qué bloque ejecutar. Si la condición interna para decidir qué bloque ejecutar. Si la condición interna es falsa, se evalúa la condición2 y se ejecuta su bloque correspondiente.

Si la condicion1 es falsa, se ejecuta el bloque else, donde también pueden evaluarse otras condiciones si existe un if adicional.

Ejemplo 9. Sentencia if anidado

```
/*Programa ejemplo sentencia if anidado.*/  
public class Ejemplo6 {  
    public static void main(String[] args) {  
        int num1 = 10;  
        int num2 = 15;  
        int num3 = 0;  
        if (num1 > 50) {  
            if (num2 < 30) {  
                num3 = num3 + 130;  
                System.out.println("num3:" + num3);  
            } else {  
                num3 = num3 - 130;  
                System.out.println("num3:" + num3);  
            }  
        }  
        else {  
            num3 = num3 = 200;  
            System.out.println("num3:" + num3);  
        }  
    }  
}
```

La sentencia switch

La sentencia switch es una estructura de decisión de múltiples alternativas, que permite ejecutar diferentes bloques de código según el valor de una variable o expresión. La expresión puede ser de tipo byte, short, int, char, una enumeración o string.

Cada case debe tener un valor constante y único, compatible con el tipo de expresión. Si ningún caso coincide, se ejecuta la opción default (si existe). Al encontrar una coincidencia las

sentencias se ejecutan hasta un `break` o hasta el final del `switch`. La forma general de la sentencia `switch` tradicional es:

```
switch(expression) {  
    case constante1:  
        secuencia de sentencias1  
        break;  
    case constante2:  
        secuencia de sentencias2  
        break;  
    case constante3:  
        secuencia de sentencias3  
        break;  
    ...  
    default:  
        secuencia de sentencias  
}
```

A continuación, se muestra el diagrama de flujo y un ejemplo.

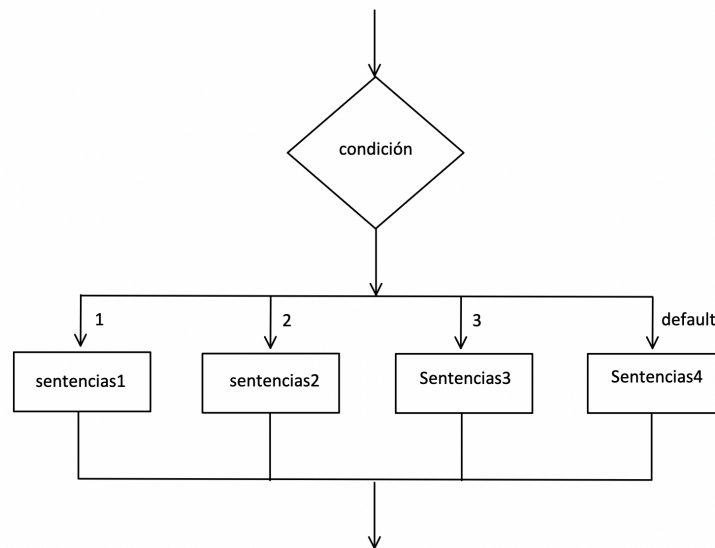


Figura 4: Diagrama de flujo de la sentencia `switch`.

**Ejemplo 10.** Sentencia switch

```
/*
 Programa ejemplo sentencia switch".
*/

public class Ejemplo7 {
    public static void main(String[] args) {
        int i = 10;
        switch (i) {
            case 10: // Coincidió la opción
                System.out.println("Diez");
                break;
            case 20:
                System.out.println("Veinte");
                break;
            default:
                System.out.println("Opcion no valida");
                break;
        }
    }
}
```

La sentencia break en un switch es opcional, pero normalmente se usa para salir del switch cuando se ejecuta un case. Si no se incluye break, el programa continúa ejecutando los casos siguientes hasta encontrar un break o llegar al final del switch, fenómeno conocido “caída”.

Incremento y decremento

Los operadores de incremento ++ y decremento – aumenta o disminuyen una variable en una unidad. Pueden usarse en modo posfijo (x++, x--) o prefijo (++x,--x).

Cuando se usan solos, ambos modos producen el mismo resultado. Sin embargo dentro de una expresión, el prefijo modifica el valor antes de usarse, mientras el posfijo primero usa el valor y luego lo modifica.

Tiene algunas propiedades especiales que los hacen bastante interesantes.

x ++; es lo mismo que. x = x + 1;

y

x --; es lo mismo que x = x - 1;



Otro ejemplo

```
int i = 7;
int j = 7;
//esto da como resultado 16, ya que primero incrementa i
int x = 2 * ++ i;
//esto da como resultado 14, ya que primero hace 2*7 y
//despues incrementa j;

int y = 2 * j ++;

++x; //prefijo
```

Ejemplo 11. Prueba de incrementos

```
// Probar el incremento prefijo y postfijo.
public class IncrementosPrueba {
    public static void main(String[] args) {
        int x, y;
        x = 10;
        y = ++x;
        System.out.println("x = " + x + " y = " + y);
        x = 10;
        y = x++;
        System.out.println("x = " + x + " y = " + y);
    }
}
```

Estructuras de Control de repetición

Un bucle es una estructura de control que repite una o varias instrucciones. En Java existen tres tipos: while, do-while y for, que se diferencian por la forma en que controlan la repetición.

Los bucles pueden ser condicionales, cuando se ejecutan mientras una condición sea verdadera y no se conoce el número de iteraciones o controlados por contador, cuando se sabe cuántas veces deben repetirse.

Ciclo for

En java, el bucle for es ideal para escribir bucles controlados por contador. Esta diseñado específicamente para inicializar, probar y actualizar una variable de control de bucle. La figura muestra la lógica de un bucle while:

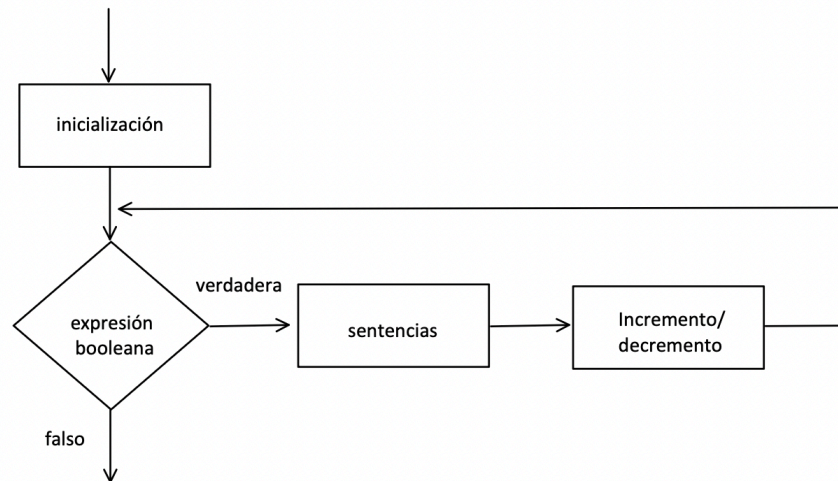


Figura 5: Diagrama de flujo de la sentencia for.

La forma general del bucle o ciclo **for** para repetir una sola instrucción es:

for(*inicialización*; *condición*; *incremento*)

sentencias;

El encabezado del bucle for contiene tres expresiones separadas por punto y coma. La **inicialización** se ejecuta una sola vez y establece la variable de control. La **condición** es una expresión booleana que determina si el bucle continúa ejecutándose. La expresión de incremento o decremento se ejecuta al final de cada iteración para actualizar la variable de control.

Ejemplo 12. Ciclo for

```
/*
Programa ejemplo ciclo for".
*/

public class ContadorFor {
    public static void main(String[] args) {
        // El encabezado for contiene inicialización;
        // condición de continuación; incremento/decremento
        for (int contador = 1; contador <= 10; contador++) {
            System.out.printf("%d ", contador);
        }
        System.out.println();
    }
}
```



Un punto importante sobre los bucles for es que la expresión condicional siempre se prueba en la parte superior del bucle. Esto significa que es posible que el código dentro del ciclo no se ejecute en absoluto si la condición es falsa para empezar.

```
for(cuenta=10; cuenta < 5; cuenta++)  
    x += cuenta; // esta declaración no se ejecutará
```

Este ciclo nunca se ejecutará porque su variable de control cuenta, es mayor que 5 cuando se ingresa al ciclo por primera vez. Esto hace que la expresión condicional, `cuenta < 5`, sea falsa desde el principio; por lo tanto, no ocurrirá ni siquiera una iteración del ciclo.

Ciclo while

El bucle while consta de una expresión booleana y un bloque de sentencias que se repite mientras la condición sea verdadera. Primero se evalúa la condición y, si es verdadera, se ejecutan las sentencias; luego la condición se vuelve a evaluar.

Este proceso se repite en cada iteración hasta que la expresión booleana sea falsa, momento en el que el bucle termina. La figura muestra la lógica de un bucle while:

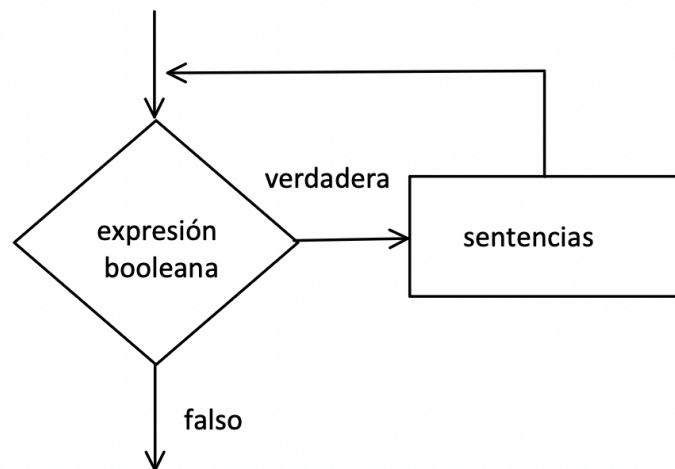


Figura 6: Diagrama de flujo de la sentencia while.

La forma generar del ciclo while es:

```
while(expresión_booleana) {
```



```
    sentencias;  
  
}
```

Ejemplo 13. Ciclo while

```
/*  
    Programa ejemplo ciclo while".  
*/  
  
public class ContadorWhile {  
    public static void main(String[] args) {  
        int cuenta = 1; // declara e inicializa la variable de control  
        while (cuenta <= 10) { // condición de continuación del ciclo  
            System.out.printf("%d ", cuenta);  
            ++cuenta; // variable de control de incremento  
        }  
        System.out.println();  
    }  
}
```

Aquí, cuenta se inicializa con el entero 1. Cada vez que pasa por el bucle se imprime la variable cuenta y luego se incrementa. Este proceso continua hasta que cuenta sea menor o igual a 10.

Al igual que el ciclo for, el while comprueba la expresión condicional en la parte superior del bucle, lo que significa que es posible que el código del ciclo no se ejecute en absoluto. Esto elimina la necesidad de realizar una prueba separada antes del ciclo.

El ciclo do-while

El bucle do-while es un ciclo de post-prueba, lo que significa que evalúa la condición después de ejecutar el bloque de instrucciones. Por esta razón, siempre se ejecuta al menos una vez, incluso si la condición es falsa desde el inicio. El ciclo continúa repitiéndose mientras la expresión booleana sea verdadera y termina cuando esta se evalúa como falsa.

La figura muestra la lógica de un bucle do-while:

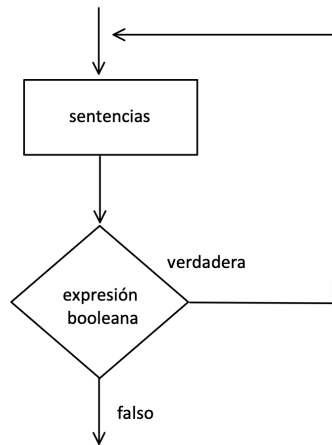


Figura 7: Diagrama de flujo de la sentencia do-while.

La forma general del ciclo do-while es:

```
do {
    sentencias;
} while(condicion);
```

Ejemplo 14. Ciclo do-while

```
/* Programa ejemplo ciclo do-while".
*/
public class ContadorDoWhile {
    public static void main(String[] args) {
        int cuenta = 1;
        do {
            System.out.printf("%d ", cuenta);
            ++cuenta;
        } while (cuenta <= 10);
        System.out.println();
    }
}
```

Acumulador

Un total acumulado es variable que guarda un total acumulado, sumando valores en cada iteración de un bucle. Se usa comúnmente junto con un bucle que lee una serie de números para calcular su suma total.



Centinela

Un centinela es un valor especial que indica el final de una lista de datos cuando no se conoce cuántos valores se ingresarán. Al detectar el valor centinela, el bucle se detiene, señalando que no hay más datos por procesar.

Ciclos anidados.

Los **bucles anidados** se utilizan cuando una operación repetitiva debe ejecutarse dentro de otra repetición. Están formados por un bucle externo y uno o más bucles internos.

El bucle interno se ejecuta completamente en cada iteración del bucle externo, y el total de iteraciones se obtiene multiplicando las iteraciones de todos los bucles.

El siguiente programa usa un ciclo for anidado para calcular la tabla del uno

Ejemplo 15. Ciclo for anidado

```
/*Ciclo anidado */  
  
public class CicloAnidado {  
    public static void main(String[] args) {  
        // ciclo externo  
        for (int i = 1; i < 2; i++) {  
            // ciclo interno  
            for (int j = 1; j <= 10; j++) {  
                System.out.printf("%d * %d = %d", i, j, i * j);  
                System.out.println();  
            }  
        }  
    }  
}
```

El ejemplo muestra un ciclo for anidado, donde el ciclo externo controla el valor de i y el ciclo interno controla el valor de j. Por cada iteración del ciclo externo, el ciclo interno se ejecuta completamente, del 1 al 10.

En este caso, el ciclo externo se ejecuta una sola vez (i=1), y el ciclo interno imprime la tabla de multiplicar del 1. Cuando el ciclo interno termina, se incrementa i, la condición del ciclo externo deja de cumplirse y el programa continúa después del ciclo.

